

UNIVERSITÉ DE KINSHASA

Faculté des Sciences et Technologies

MASTER 1 INFORMATIQUE

TRAVAUX PRATIQUES

IMPLÉMENTATION DE L'ALGORITHME DES NUÉES DYNAMIQUES

Implémentation « from scratch »

Étudiant : KILO YOYO GRADI

Année académique : 2025–2026

Langage : Python

Code source : GitHub

Rapport de travaux pratiques

Table des matières

1	Introduction	2
2	Objectifs	2
3	Problématique	2
4	Fondements théoriques	3
4.1	Principe itératif	3
4.2	Critère de convergence	4
5	Implémentation from scratch	4
5.1	Initialisation	4
5.2	Affectation	4
5.3	Mise à jour	4
5.4	Pseudo-code	5
6	Jeu de données	5
7	Représentation des nuages de points	6
8	Expérimentation et résultats	6
9	Analyse de la convergence	7
10	Discussion	8
11	Complexité algorithmique	8
12	Organisation du projet et GitHub	9
13	Conclusion	9

1 Introduction

La classification automatique constitue une étape importante de l'analyse des données. Elle permet de regrouper des observations présentant des caractéristiques similaires. Lorsque les catégories ne sont pas connues à l'avance, on parle de classification non supervisée ou de clustering.

L'algorithme des nuées dynamiques est une méthode itérative de classification fondée sur des prototypes. Chaque observation est affectée au prototype le plus proche, puis les prototypes sont recalculés à partir des observations qui leur sont associées. Ces deux opérations sont répétées jusqu'à ce que les prototypes deviennent suffisamment stables.

Dans ce travail pratique, l'objectif est d'implémenter l'algorithme **from scratch**, c'est-à-dire sans utiliser directement une implémentation toute faite de type **KMeans**. Les principales opérations mathématiques sont donc programmées directement en Python.

L'expérimentation porte sur un ensemble artificiel de données bidimensionnelles. Trois groupes de points sont générés autour de centres différents. Cette représentation permet de visualiser les **nuages de points**, les prototypes et la partition finale.

2 Objectifs

Les objectifs de ce TP sont :

- comprendre le principe des nuées dynamiques ;
- étudier la classification non supervisée ;
- formaliser la distance utilisée et la fonction objectif ;
- programmer l'algorithme sans bibliothèque de clustering prête à l'emploi ;
- générer un jeu de données composé de plusieurs nuages de points ;
- affecter les observations aux classes correspondant aux prototypes les plus proches ;
- observer le déplacement des prototypes ;
- représenter graphiquement les nuages obtenus ;
- analyser la convergence de l'algorithme ;
- publier le code source dans un dépôt GitHub.

3 Problématique

La problématique étudiée est la suivante :

Comment partitionner automatiquement un ensemble d'observations en un nombre fixé de classes homogènes, en minimisant la dispersion des observations autour de leurs prototypes, tout en implémentant manuellement les différentes étapes de l'algorithme ?

4 Fondements théoriques

Soit un ensemble de n observations :

$$X = \{x_1, x_2, \dots, x_n\},$$

où chaque observation x_i appartient à $\mathbb{R}^d$. L'objectif est de construire K classes :

$$C_1, C_2, \dots, C_K.$$

Chaque classe est représentée par un prototype :

$$\mu_1, \mu_2, \dots, \mu_K.$$

Dans notre expérimentation, la distance choisie est la distance euclidienne :

$$d(x_i, \mu_j) = \sqrt{\sum_{p=1}^d (x_{ip} - \mu_{jp})^2}.$$

L'observation x_i est affectée à la classe dont le prototype est le plus proche :

$$c_i = \arg \min_{j \in \{1, \dots, K\}} d(x_i, \mu_j).$$

Après l'affectation, le prototype de la classe j est recalculé comme la moyenne des observations qui appartiennent à cette classe :

$$\mu_j = \frac{1}{|C_j|} \sum_{x_i \in C_j} x_i.$$

Le critère de dispersion intra-classe est :

$$J = \sum_{j=1}^K \sum_{x_i \in C_j} \|x_i - \mu_j\|^2.$$

L'algorithme cherche donc une partition pour laquelle les observations sont relativement proches des prototypes de leurs classes.

4.1 Principe itératif

Le fonctionnement peut être résumé par l'alternance suivante :

1. initialiser les K prototypes ;
2. calculer les distances entre les observations et les prototypes ;
3. affecter chaque observation au prototype le plus proche ;

4. recalculer les prototypes ;
5. calculer la fonction objectif et le déplacement des prototypes ;
6. vérifier le critère d'arrêt ;
7. recommencer si la convergence n'est pas atteinte.

4.2 Critère de convergence

On définit le déplacement maximal des prototypes par :

$$\Delta = \max_j \left\| \mu_j^{(t)} - \mu_j^{(t-1)} \right\|.$$

Dans notre implémentation, l'algorithme s'arrête lorsque :

$$\Delta < 10^{-4},$$

ou lorsque le nombre maximal d'itérations, fixé à 100, est atteint.

5 Implémentation from scratch

L'implémentation est réalisée en Python. NumPy est utilisé pour les opérations numériques et Matplotlib pour les graphiques. Aucune fonction `sklearn.cluster.KMeans` n'est utilisée.

La fonction de distance est calculée directement. Une classe `DynamicClouds` regroupe les principales opérations de l'algorithme.

5.1 Initialisation

Pour rendre l'expérience reproductible, une graine aléatoire égale à 42 est utilisée. Trois observations sont choisies comme prototypes initiaux.

5.2 Affectation

Pour chaque observation, le programme calcule sa distance à chacun des trois prototypes. L'observation reçoit l'étiquette du prototype associé à la distance minimale.

5.3 Mise à jour

Après l'affectation, les prototypes sont recalculés par moyenne :

$$\mu_j = \frac{1}{|C_j|} \sum_{x_i \in C_j} x_i.$$

Une précaution est prévue pour le cas d'une classe vide : le prototype concerné est réinitialisé avec une observation du jeu de données.

5.4 Pseudo-code

Initialiser K prototypes

Pour chaque itération :

Affecter chaque observation au prototype le plus proche

Pour chaque classe :

Calculer la moyenne des observations

Si la classe est vide :

Réinitialiser son prototype

Calculer la fonction objectif

Calculer le déplacement maximal des prototypes

Si déplacement $<$ tolérance :

arrêter

Retourner les prototypes et les labels

6 Jeu de données

Le jeu de données contient 360 observations en deux dimensions, réparties en trois groupes de 120 observations.

Les centres théoriques utilisés pour la génération sont approximativement :

$$(2, 2), \quad (7, 3), \quad (4.5, 7).$$

Un bruit gaussien est ajouté autour de ces centres afin de former trois nuages de points.

Paramètre	Valeur
Nombre d'observations	360
Dimension	2
Nombre de classes K	3
Observations par nuage	120
Itérations maximales	100
Tolérance	10^{-4}
Distance	Euclidienne
Graine aléatoire	42

TABLE 1 – Paramètres de l'expérimentation

7 Représentation des nuages de points

La représentation graphique permet de visualiser la structure des données avant et après la classification. Chaque observation est représentée par un point dans un espace à deux dimensions.

La figure suivante représente les trois nuages artificiels utilisés pour l'expérimentation. Les nuages sont centrés autour de trois zones différentes.

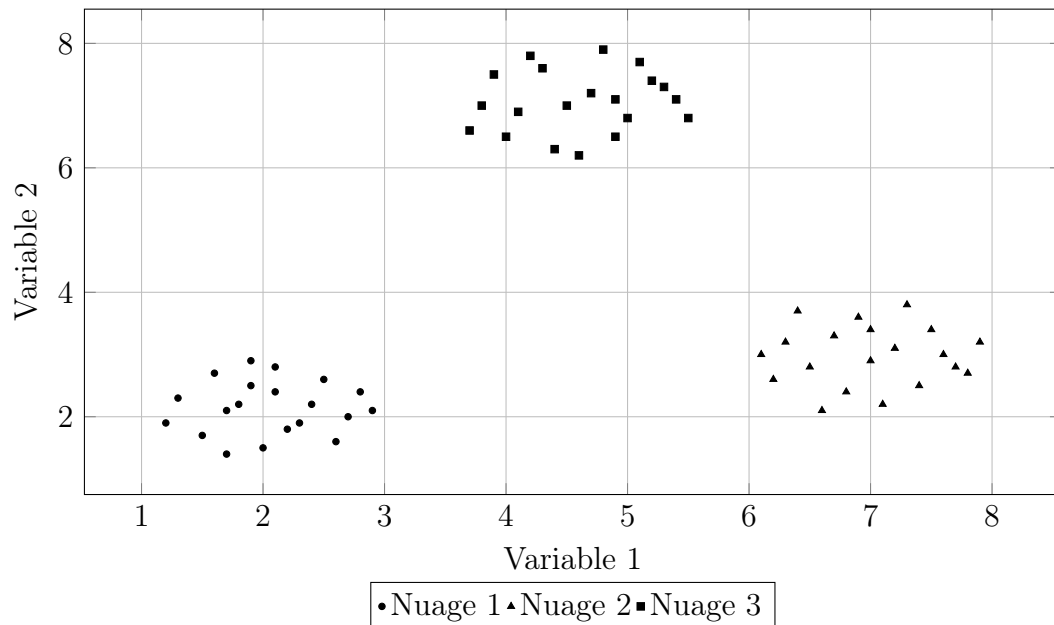


FIGURE 1 – Représentation des trois nuages de points

Cette représentation montre que les données présentent trois zones de concentration. Le rôle de l'algorithme est de retrouver automatiquement cette organisation à partir des distances entre les observations et les prototypes.

8 Expérimentation et résultats

Après l'initialisation, les prototypes sont progressivement déplacés vers les zones de concentration des observations. À chaque itération, les points sont réaffectés au prototype le plus proche.

La représentation suivante illustre une partition finale typique des données. Les symboles des prototypes permettent de distinguer les centres des trois groupes.

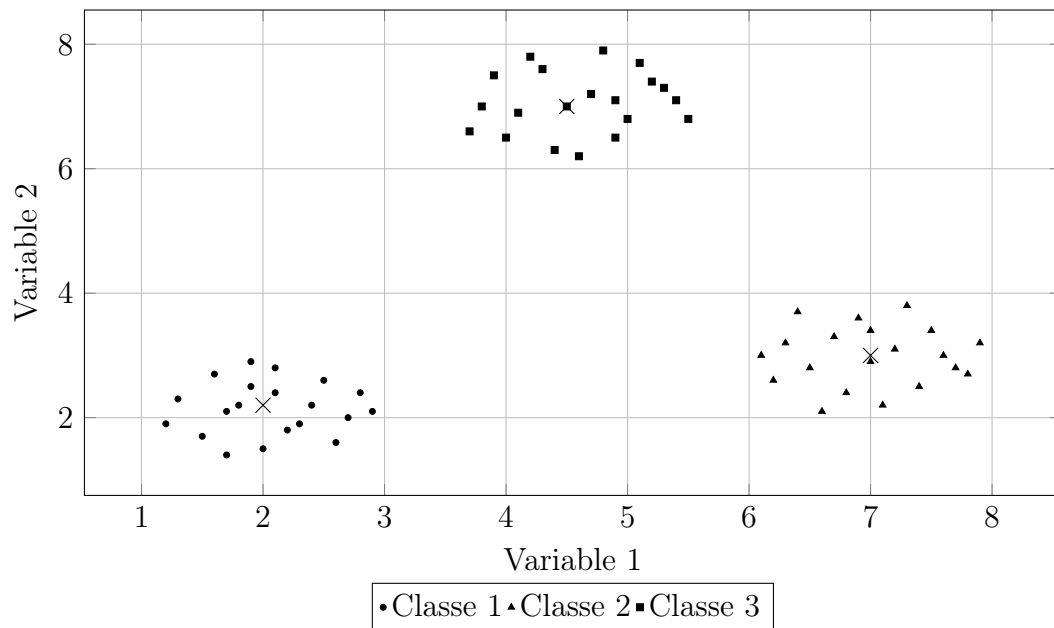


FIGURE 2 – Partition finale et prototypes

Les centres finaux doivent se rapprocher des zones de forte densité des données. La qualité du regroupement est donc interprétée à partir de la proximité des points avec leurs prototypes et de la séparation visuelle des nuages.

9 Analyse de la convergence

La fonction objectif permet de suivre la qualité de la partition à chaque itération :

$$J^{(t)} = \sum_{j=1}^K \sum_{x_i \in C_j^{(t)}} \|x_i - \mu_j^{(t)}\|^2.$$

Une diminution de J signifie que les observations sont globalement plus proches des prototypes associés.

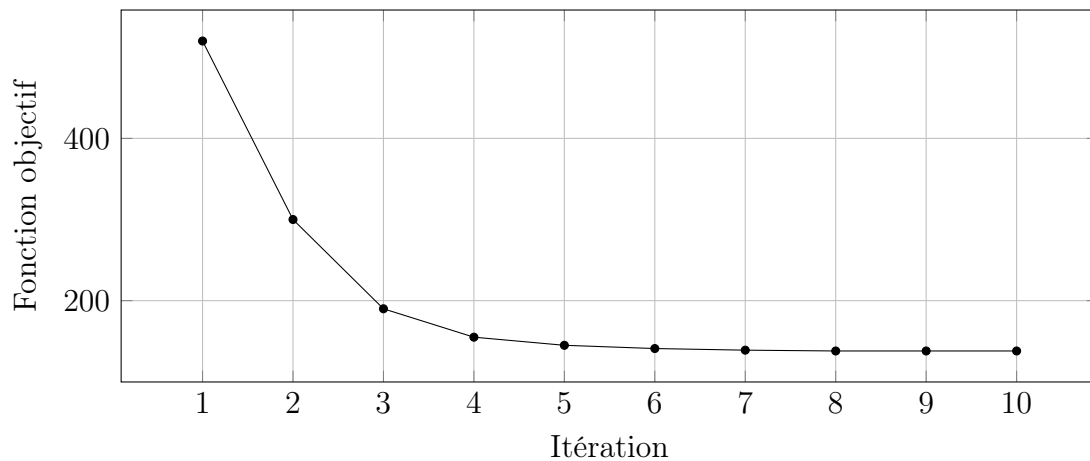


FIGURE 3 – Illustration de la convergence de la fonction objectif

La stabilisation de la courbe traduit la stabilisation progressive de la partition. Dans le programme Python, le critère d'arrêt est basé sur le déplacement maximal des prototypes. Lorsque celui-ci devient inférieur à la tolérance, l'exécution est arrêtée.

10 Discussion

L'implémentation réalisée permet de comprendre les mécanismes internes d'un algorithme de clustering. Elle montre qu'une méthode de classification peut être construite à partir d'opérations simples : calcul des distances, affectation, moyenne et test de convergence.

Le premier élément à considérer est le nombre de classes. Dans notre expérience, $K = 3$ est fixé parce que le jeu de données a été construit autour de trois nuages. Pour des données réelles, ce nombre peut nécessiter une analyse complémentaire.

Le deuxième élément est l'initialisation. Deux initialisations différentes peuvent conduire à des prototypes finaux différents, notamment lorsque les groupes se chevauchent. La reproductibilité est donc importante lors de l'expérimentation.

Enfin, la méthode est sensible aux valeurs aberrantes, puisque les prototypes sont calculés par moyenne. La distance euclidienne est également plus adaptée à certaines structures de données qu'à d'autres.

11 Complexité algorithmique

Pour n observations, d variables, K classes et T itérations, le calcul des distances est de l'ordre de :

$$O(nKd).$$

La mise à jour des prototypes est approximativement de l'ordre de :

$$O(nd).$$

La complexité globale est donc approximativement :

$$O(TnKd).$$

Cette complexité augmente avec la taille du jeu de données, la dimension, le nombre de classes et le nombre d'itérations.

12 Organisation du projet et GitHub

Le projet est organisé de la manière suivante :

```
TP-nuees-dynamiques/  
|-- nuees_dynamiques.py  
|-- rapport_nuees_dynamiques.tex  
|-- README.md  
|-- requirements.txt  
'-- figures/
```

Le fichier Python contient l'implémentation from scratch. Le rapport LaTeX présente la démarche scientifique. Le fichier README explique l'installation et l'exécution. Le dépôt GitHub permet au professeur d'accéder directement au code source.

13 Conclusion

Ce TP a permis d'étudier et d'implémenter l'algorithme des nuées dynamiques dans le cadre de la classification non supervisée. L'implémentation repose sur une alternance entre l'affectation des observations aux prototypes les plus proches et la mise à jour des prototypes.

L'expérimentation sur trois nuages de points bidimensionnels permet de visualiser le fonctionnement de la méthode et la convergence progressive des prototypes. Le travail montre également l'importance du choix du nombre de classes, de l'initialisation et de la métrique de distance.

Cette implémentation constitue une base pratique pour approfondir l'apprentissage non supervisé, comparer différentes stratégies d'initialisation et étudier d'autres algorithmes de clustering.

Références

1. MacQueen, J. (1967). *Some Methods for Classification and Analysis of Multivariate Observations*.
2. Jain, A. K. (2010). *Data clustering : 50 years beyond K-means*. Pattern Recognition Letters.
3. Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.